DATABASE MANAGEMENT SYSTEMS

CT 652 | BE / BCT | Year III / Part II

Tribhuvan University — Institute of Engineering

CHAPTER 3: RELATIONAL LANGUAGES & RELATIONAL MODEL

Question Bank with Comprehensive Answers — RA, TRC, DRC, SQL, Views, Triggers, Embedded SQL

SYLLABUS OVERVIEW — Unit 3

Subject: CT 652 | Full Marks: 80 | Pass Marks: 32 | Duration: 3 hrs

Unit	Topic	Hours	Marks
1	Introduction	3 hrs	4
2	Data Models	7 hrs	12
3	Relational Languages and Relational Model	7 hrs	12
4	Database Constraints and Normalization	6 hrs	12
5	Query Processing and Optimization	4 hrs	8
6	File Structure and Hashing	4 hrs	8
7	Transactions Processing and Concurrency Control	6 hrs	12
8	Crash Recovery	4 hrs	6
9	Advanced Database Concepts	4 hrs	6
Total		**45 hrs**	**80**

Chapter 3 Topics (7 Hours | 12 Marks)

3.1 Relational Data Model
3.2 Integrity Constraints
3.3 Relational Algebra
3.4 Tuple Relational Calculus (TRC)
3.5 Domain Relational Calculus (DRC)
3.6 SQL
3.7 DDL, DML in SQL
3.8 Views in SQL
3.9 Embedded SQL
3.10 Dynamic SQL

STANDARD RELATIONS USED IN EXAM QUESTIONS

Most Chapter 3 questions use two standard sets of relations. These are defined here for reference throughout the document.

Set A — Customer / Agent / Orders

```
Customer(cid, cname, city, rating, sal)
Agent   (aid, aname, city, percent)
Orders  (oid, month, cid, aid, qty, amount)

Sample data:
Customer: c001 Nick    London  8  50000
          c002 Alice   Shimla  6  40000
          c003 Bob     London  9  60000
          c004 Carol   Bangalore 7 45000
Agent:    a01  Smith   London  12
          a02  Jones   New York 10
          a03  Brown   London  11
Orders:   o01  Jan c001 a01 500 10000
          o02  Feb c002 a02 300  6000
          o03  Mar c001 a03 700 14000
```

Set B — Employee / Department

```
Employee  (empid/eid, name/ename, dept_id/deptno, salary, hiredate, city,
job_title)
Department(dept_id/deptno, dept_name/dname, location/city)

Sample data:
Employee: e01 Alice  d01 55000  2018-03-01  Kathmandu  Engineer
          e02 Bob    d02 45000  2020-06-15  Pokhara    Analyst
          e03 Carol  d01 70000  2015-01-10  Kathmandu  Manager
          e04 David  d03 30000  2022-09-01  Lalitpur   Clerk
Department: d01 Sales      Kathmandu
            d02 Research   Pokhara
            d03 Finance    Lalitpur
            d04 HR         Kathmandu
```

CHAPTER 3 — Conceptual Questions & Answers

Q: Define the relational data model. Explain: relation, schema, instance, tuple, attribute, domain, degree, cardinality.
Asked: 2074 Bhadra; 2072 Magh; 2080 Ashwin; 2076 Baisakh | Marks: [4]

Answer

Relational Data Model

The relational data model, proposed by E.F. Codd in 1970, organizes data as a collection of two-dimensional tables called relations. It provides a simple, yet powerful, abstraction based on the mathematical concept of a relation (set theory). It is the most widely used data model in practice.

Core Terminology

Term	Definition	Example
Relation	A two-dimensional table consisting of rows and columns; represents an entity set or relationship	Employee table, Department table
Relation Schema	The logical design/structure of a relation — specifies the name of the relation and its attributes with their domains; written as R(A1, A2, ..., An)	Employee(empid, name, deptno, salary)
Relation Instance	The current set of rows (tuples) in a relation at a given point in time; changes as data is inserted, deleted, or updated	The actual rows currently in the Employee table
Tuple	A single row in a relation; represents one entity or relationship instance	(e01, Alice, d01, 55000)
Attribute	A named column of a relation; represents a property of the entity	empid, name, salary
Domain	The set of permitted atomic (indivisible) values for an attribute	dom(salary) = non-negative integers; dom(city) = strings of up to 50 chars
Degree	The number of attributes (columns) in a relation schema	Degree of Employee(empid, name, dept, salary) = 4
Cardinality	The number of tuples (rows) currently in a relation instance	If Employee has 50 rows, cardinality = 50
Null	A special marker representing an unknown, missing, or inapplicable value; distinct from zero or empty string	NULL salary means salary is unknown

Relational Model vs ER Model

Aspect	ER Model	Relational Model
Purpose	Conceptual design (what to store)	Implementation model (how to store)
Constructs	Entities, attributes, relationships	Relations (tables), tuples, attributes
Primary Key	Underlined attribute in ER diagram	Enforced via PRIMARY KEY constraint

Relationships	Diamond shapes / relationship sets	Foreign keys linking tables
Representation	ER Diagram (graphical)	Table schema (textual/logical)
Weak Entity	Double rectangle + discriminator	Separate table; PK = owner PK + discriminator
Phase	Database design (before creation)	Database implementation (in DBMS)

Q: What are the different integrity constraints in the relational model? Explain each with examples.
Asked: 2080 Ashwin (Back Exam) | Marks: [4]

Answer

Integrity Constraints in the Relational Model

Integrity constraints are rules that ensure the accuracy, validity, and consistency of data in a database. They are enforced by the DBMS automatically.

1. Domain Constraint

Specifies that the value of each attribute must be an atomic value from its defined domain. No multi-valued or composite attributes are allowed in a single column.

```
salary INTEGER CHECK (salary >= 0)
age INTEGER CHECK (age BETWEEN 0 AND 150)
gender CHAR(1) CHECK (gender IN ('M', 'F', 'O'))
```

2. Key Constraint (Uniqueness Constraint)

Each tuple in a relation must be uniquely identifiable. The primary key attribute(s) must be unique for all tuples. No two tuples can have the same primary key value.

```
empid INT PRIMARY KEY   -- no two employees can have same empid
UNIQUE (email)          -- email also unique (candidate key)
```

3. Entity Integrity Constraint

The primary key attribute(s) of a relation cannot be NULL. Each entity must be identifiable — if its primary key is unknown, it cannot be stored.

```
empid INT NOT NULL PRIMARY KEY   -- empid can never be NULL
-- This ensures every Employee tuple has a valid, unique identifier
```

4. Referential Integrity Constraint

A foreign key value in a relation either must match an existing primary key value in the referenced relation, or must be NULL (if the foreign key is not part of the primary key and nulls are allowed). This prevents 'dangling references'.

```
dept_id INT REFERENCES Department(dept_id)   -- FK constraint
-- If dept_id = 'd99' in Employee but d99 does not exist in Department,
-- the DBMS will reject the insertion (referential integrity violation)
```

Enforcement actions on deletion/update of referenced tuple:

- CASCADE: Delete/update all referring tuples automatically
- SET NULL: Set the FK value to NULL in referring tuples
- SET DEFAULT: Set the FK value to its default value
- RESTRICT / NO ACTION: Reject the delete/update if it would violate constraint

5. Semantic Integrity Constraints (Business Rules)

Additional user-defined constraints that enforce business logic beyond structural constraints. Implemented via CHECK constraints, ASSERTIONS, or TRIGGERS.

```
CHECK (salary >= 10000)              -- business rule: minimum salary
CHECK (end_date > start_date)         -- logical consistency
```

Q: What is relational algebra? Discuss the fundamental relational algebra operations with examples.

Asked: 2081 Chaitra; 2079 Ashwin; 2075 Bhadra; 2070–2077 (all years) | Marks: [2+4 to 6]

Answer

Relational Algebra

Relational algebra is a procedural query language that operates on relations (tables) and produces relations as output. It was introduced by E.F. Codd as the theoretical basis for SQL. Each operation takes one or two relations as input and returns a new relation as output. Queries are expressed as a sequence of operations.

The result of every RA operation is a relation, making operations composable (output of one can be input to another).

8 Fundamental Operations

1. Selection (σ — Sigma)

Selects tuples (rows) from a relation that satisfy a given predicate/condition. Horizontal filtering.

```
Syntax:  σ<condition>(R)
Example: σ salary>50000 (Employee)            -- employees earning > 50,000
Example: σ dept='Sales' AND city='Kathmandu' (Employee)  -- Sales dept in
Kathmandu
```

2. Projection (π — Pi)

Selects specific attributes (columns) from a relation, eliminating duplicates. Vertical filtering.

```
Syntax:  π<attribute list>(R)
Example: π name, salary (Employee)            -- only name and salary columns
Example: π cname, city (Customer)             -- customer names and cities
```

3. Union (∪)

Returns all tuples that appear in either of two union-compatible relations (same arity, same domains). Duplicates are eliminated.

```
Syntax:  R ∪ S
Condition: R and S must be union-compatible (same degree; same attribute
domains)
Example: π city(Customer) ∪ π city(Agent)
         -- all cities where customers OR agents are located
```

4. Set Difference (−)

Returns tuples that appear in the first relation but NOT in the second. Must be union-compatible.

```
Syntax:  R − S
Example: π cid(Customer) − π cid(Orders)
         -- customers who have NEVER placed any order
Example: π aid(Agent) − π aid(Orders)
         -- agents who have never handled any order
```

5. Cartesian Product (×)

Combines every tuple of relation R with every tuple of relation S. If R has m tuples and n attributes, S has p tuples and q attributes, result has m×p tuples and n+q attributes. Used as basis for joins.

```
Syntax:  R × S
Example: Employee × Department
         -- all possible combinations of employee and department tuples
         -- (rarely useful alone; used with selection to form joins)
```

6. Rename (ρ — Rho)

Renames a relation or its attributes. Useful for self-joins and disambiguation.

```
Syntax:  ρ<new_name>(R)    or    ρ<new_name>(A1, A2, ...)(R)
Example: ρ Emp1(Employee)   -- rename Employee to Emp1
Example: ρ Manager(empid→mgrid, name→mgrname)(Employee)
```

<u>Derived (Additional) Operations</u>

7. Natural Join (□)

Combines two relations by matching on all common attribute names, keeping only one copy of each common attribute. Automatically finds common attributes and equates them.

```
Syntax:  R □ S
Example: Employee □ Department
         -- joins on common attribute deptno; shows each employee with their
dept name
Result attributes: empid, name, deptno, salary, dname, location
```

8. Division (÷)

Used to express 'for all' queries. R ÷ S returns tuples from R that are associated with ALL tuples in S.

```
Syntax:  R ÷ S
Example: Find agents who have placed orders for ALL customers:
         π aid,cid(Orders) ÷ π cid(Customer)
         -- returns aids of agents who have orders with every single customer
```

Quick Summary Table

Operation	Symbol	Type	Purpose
Selection	σ	Unary	Filter rows based on condition
Projection	π	Unary	Select specific columns; remove duplicates
Union	∪	Binary	All tuples from both relations (no duplicates)
Set Difference	−	Binary	Tuples in R not in S
Cartesian Product	×	Binary	All combinations of tuples from R and S
Rename	ρ	Unary	Rename relation or attributes
Natural Join	□	Binary	Equi-join on common attributes
Division	÷	Binary	'For all' queries — tuples in R associated with all of S

Q: Differentiate between relational algebra and relational calculus. Also distinguish TRC, DRC, and RA.

Asked: 2079 Chaitra; 2075 Baisakh | Marks: [4]

Answer

Relational Algebra vs Relational Calculus

Aspect	Relational Algebra (RA)	Relational Calculus (RC)
Nature	Procedural — specifies HOW to retrieve data (step-by-step operations)	Declarative — specifies WHAT data to retrieve (no step-by-step)
Basis	Set theory and operations on relations	Predicate logic / First-order logic
Query style	Sequence of algebra operations (σ, π, □, ∪ , −, ×)	Logical formula / predicate expression
Completeness	Forms the basis — RC queries can be translated to RA	Relationally complete if it can express all RA queries
Ease of use	More mechanical; good for optimization	More natural; closer to English language
Type	RA itself has no sub-types (but additional ops are derived)	Two sub-types: TRC (tuple variables) and DRC (domain variables)
Equivalence	RA and RC (both TRC and DRC) are equivalent in expressive power	RC queries can be translated to equivalent RA and vice versa
Usage in practice	Foundation for SQL query optimization (query plans)	Foundation for SQL query language design

TRC vs DRC

Aspect	Tuple Relational Calculus (TRC)	Domain Relational Calculus (DRC)
Variable type	Tuple variables ranging over entire tuples of a relation	Domain variables ranging over individual attribute values
Notation	{ t | P(t) } — t is a tuple variable	{ <x1,x2,...> | P(x1,x2,...) } — xi are domain variables
Attribute access	t.attribute (dot notation)	Use the domain variable directly for each attribute
Granularity	Whole tuple at a time	Individual attribute values
Basis for SQL	SQL SELECT...FROM...WHERE is closest to TRC	QBE (Query by Example) is based on DRC
Safety	Must be restricted to safe expressions (finite result)	Must be restricted to safe expressions

---

Q: Write relational algebra expressions for queries on Employee(empid, name, dept, salary, city) and Department(deptno, dname, city).
Asked: 2080 Ashwin; 2079 Ashwin; 2079 Chaitra; 2078 Baishakh; 2073 Magh — multiple years | Marks: [4–6]

Answer

Relation Schemas

```
Employee  (empid, name, deptno, salary, city)
Department(deptno, dname, city)
```

RA Expressions

Q1: Names of employees working in 'Sales' department in city 'Kathmandu'

```
Step 1: Find deptno of 'Sales':  π deptno (σ dname='Sales' (Department))
Step 2: Find employees in that dept AND city='Kathmandu':
        π name (σ city='Kathmandu' (Employee □ (π deptno (σ dname='Sales'
(Department)))))

Or equivalently:
        π name (σ city='Kathmandu' AND dname='Sales' (Employee □ Department))
```

Q2: Names of employees with salary ≥ 10000

```
π name (σ salary >= 10000 (Employee))
```

Q3: Names of employees in same city as Department 'Research'

```
Step 1: Find city of 'Research' dept:  π city (σ dname='Research'
(Department))
```

```
Step 2: Rename: ρ ResCity(city) (π city (σ dname='Research' (Department)))
Step 3: Find employees in that city:
        π name (Employee □(city=city) ResCity)

Full expression:
π name (σ Employee.city = D.city
  (Employee × (ρ D (π city (σ dname='Research' (Department))))))
```

Q4: Names of employees in 'Finance' department

```
π name (σ dname='Finance' (Employee □ Department))
```

Q5: Employee names and department names (all employees with their depts)

```
π name, dname (Employee □ Department)
  -- natural join on deptno; only employees who have a matching department
```

Q6: Employee names and dept names — all departments listed (outer join concept in RA)

```
-- Full outer join is not a standard RA operator; use union of join and non-
matching:
π name, dname (Employee □ Department)
∪
π dname (Department − π dname (Employee □ Department))  -- depts with no
employees

-- Or in SQL: LEFT OUTER JOIN
```

Q7: Total salary per department

```
-- Aggregation is not part of basic RA; expressed using extended RA with F
(aggregate function):
deptno F SUM(salary) (Employee)
-- Result: (deptno, SUM_salary)

-- For names: deptno, dname F SUM(salary) (Employee □ Department)
```

Q8: Employees hired in last 5 years with salary > average salary

```
-- Average salary requires aggregation (extended RA):
AvgSal ← F AVG(salary) (Employee)   -- gives one tuple: avg_salary

-- Employees hired after (CURRENT_YEAR - 5) with salary > avg:
π name, salary (σ hiredate > '2020-01-01' AND salary > avg_salary
                   (Employee × AvgSal))
```

Q: Write relational algebra expressions for queries on Customer(cid,cname,city,rating,sal), Agent(aid,aname,city,percent), Orders(oid,month,cid,aid,qty,amount).

Asked: 2078 Chaitra; 2077 Chaitra; 2075 Bhadra; 2070–2078 (all years) | Marks: [4]

Answer

RA Expressions — Customer / Agent / Orders

Q1: Find customers in 'London' with rating > 5

```
σ city='London' AND rating>5 (Customer)
-- For names only: π cname (σ city='London' AND rating>5 (Customer))
```

Q2: Customers in 'London' AND 'New York' with rating > 5 (using set operations)

```
London_Customers  ← π cid (σ city='London'   AND rating>5 (Customer))
NewYork_Customers ← π cid (σ city='New York' AND rating>5 (Customer))
Result ← London_Customers ∪ NewYork_Customers

-- For complete customer info:
π cname,city,rating (σ (city='London' OR city='New York') AND rating>5
(Customer))
```

Q3: Names of agents who placed orders for customers in 'London'

```
Step 1: Find cids of London customers: LondonCids ← π cid (σ city='London'
(Customer))
Step 2: Find aids in Orders for those cids:
        LondonOrders ← π aid (Orders □ LondonCids)
Step 3: Get agent names: π aname (Agent □ LondonOrders)

Combined: π aname (Agent □ π aid (Orders □ π cid (σ city='London'
(Customer))))
```

Q4: Names of agents who have NOT placed any order for customer 'Nick'

```
Step 1: cid of Nick: NickCid ← π cid (σ cname='Nick' (Customer))
Step 2: aids who DID place orders for Nick: π aid (Orders □ NickCid)
Step 3: All agents minus agents who placed for Nick:
        π aid (Agent) − π aid (Orders □ NickCid)
Step 4: Get names: π aname (Agent □ (π aid(Agent) − π aid(Orders □ NickCid)))
```

Q5: Agents who placed orders for ALL customers (Division)

```
AllCustomers ← π cid (Customer)
AgentOrders  ← π aid, cid (Orders)
Result ← AgentOrders ÷ AllCustomers
-- Returns aids of agents who have an order for every customer
```

```
For agent names: π aname (Agent □ (AgentOrders ÷ AllCustomers))
```

Q6: Names of agents who have NOT placed orders for customers in 'Shimla'

```
ShimlaCids    ← π cid (σ city='Shimla' (Customer))
AgentsForShimla ← π aid (Orders □ ShimlaCids)
AllAgents     ← π aid (Agent)
AgentsNotForShimla ← AllAgents − AgentsForShimla
π aname (Agent □ AgentsNotForShimla)
```

Q: Define Tuple Relational Calculus (TRC). Write TRC expressions for various queries on Customer, Agent, Orders.

Asked: 2079 Jestha; 2078 Chaitra; 2077; 2076; 2073 Bhadra; 2071 Bhadra | Marks: [4]

Answer

Tuple Relational Calculus (TRC)

Tuple Relational Calculus (TRC) is a non-procedural (declarative) query language based on predicate logic. In TRC, queries are expressed as:

```
{ t | P(t) }
  where: t  = a tuple variable (ranging over tuples of a named relation)
         P(t) = a predicate (condition) that t must satisfy
         t.A = accessing attribute A of tuple variable t
```

The result is the set of all tuples t that make the predicate P(t) true.

Logical Connectives and Quantifiers in TRC

Symbol	Meaning	SQL Equivalent
∧	AND	AND
∨	OR	OR
¬	NOT	NOT
□ t □ R (P(t))	There EXISTS a tuple t in R such that P(t) is true	EXISTS subquery
∀ t ∈ R (P(t))	FOR ALL tuples t in R, P(t) is true	NOT EXISTS (NOT EXISTS) — universal quantification
t.A	Value of attribute A for tuple variable t	t.columnname (in SQL, tablealias.column)

TRC Expressions

Q1: Names of all customers from 'Bangalore'

```
{ t.cname | Customer(t) ∧ t.city = 'Bangalore' }

-- Read: the set of t.cname for all tuples t in Customer where city =
'Bangalore'
```

Q2: Names and cities of customers with rating > 5

```
{ t.cname, t.city | Customer(t) ∧ t.rating > 5 }
```

Q3: Names of agents who have placed at least one order (any customer)

```
{ t.aname | Agent(t) ∧ ∃ o (Orders(o) ∧ o.aid = t.aid) }

-- Read: agents for whom there EXISTS an order tuple o with matching aid
```

Q4: Agent names for customers from 'London' (without JOIN keyword)

```
{ t.aname | Agent(t) ∧
   ∃ o (Orders(o) ∧ o.aid = t.aid ∧
     ∃ c (Customer(c) ∧ c.cid = o.cid ∧ c.city = 'London')) }

-- Read: agents t for whom there exists an order o placed by a London customer
c
```

Q5: Names of customers from 'Bangalore' (short form)

```
{ t.cname | Customer(t) ∧ t.city = 'Bangalore' }
```

Q6: Agents who placed orders for ALL customers (Universal Quantifier)

```
{ t.aname | Agent(t) ∧
   ∀ c (Customer(c) ⇒
     ∃ o (Orders(o) ∧ o.aid = t.aid ∧ o.cid = c.cid)) }

-- Read: agent t such that FOR ALL customers c, there EXISTS an order o
--       linking t to c

-- Equivalent using ¬□¬ (double negation for universal):
{ t.aname | Agent(t) ∧
   ¬ ∃ c (Customer(c) ∧
     ¬ ∃ o (Orders(o) ∧ o.aid = t.aid ∧ o.cid = c.cid)) }

-- Read: agents t for whom there is NO customer c without an order from t
```

Q7: Names of agents who have NOT placed any order for customer 'Nick'

```
{ t.aname | Agent(t) ∧
   ¬ ∃ o (Orders(o) ∧ o.aid = t.aid ∧
     ∃ c (Customer(c) ∧ c.cid = o.cid ∧ c.cname = 'Nick')) }
```

Q: Define Domain Relational Calculus (DRC). Write DRC expressions for various queries.

Asked: 2079 Jestha; 2078 Chaitra; 2077 Chaitra; 2076 Baisakh | Marks: [4]

Answer

Domain Relational Calculus (DRC)

Domain Relational Calculus (DRC) is a non-procedural query language in which queries are expressed using domain variables that range over the domains of individual attributes (rather than over entire tuples as in TRC). QBE (Query By Example) is based on DRC.

```
General form: { <x1, x2, ..., xn> | P(x1, x2, ..., xn) }
  where: x1, x2, ..., xn = domain variables (one per attribute value needed)
         P = predicate involving domain variables
  R(x1, x2, ..., xk) means 'the tuple <x1,...,xk> exists in relation R'
```

DRC Expressions

Q1: Names and cities of customers with rating > 5

```
{ <cn, ct> | ∃ ci ∃ r ∃ s (Customer(ci, cn, ct, r, s) ∧ r > 5) }

-- where: ci=cid, cn=cname, ct=city, r=rating, s=sal
-- Read: pairs <cn, ct> such that a Customer tuple with those values exists
and r > 5
```

Q2: Names of customers from 'Bangalore'

```
{ <cn> | □ ci □ r □ s (Customer(ci, cn, 'Bangalore', r, s)) }

-- The city attribute is fixed to 'Bangalore' directly in the tuple pattern
```

Q3: Agent names for customers in 'London'

```
{ <an> | ∃ ai ∃ ac ∃ ap (Agent(ai, an, ac, ap) ∧
           ∃ oi ∃ m ∃ ci ∃ q ∃ amt (Orders(oi, m, ci, ai, q, amt) ∧
             □ cn □ r □ s (Customer(ci, cn, 'London', r, s)))) }

-- where: ai=aid, an=aname; Orders.cid=ci matches Customer.cid=ci in 'London'
```

Q4: Names of all agents who placed some order for every customer

```
{ <an> | ∃ ai ∃ ac ∃ ap (Agent(ai, an, ac, ap) ∧
   ∀ ci (( ∃ cn ∃ ct ∃ r ∃ s (Customer(ci, cn, ct, r, s))) ⇒
     □ oi □ m □ q □ amt (Orders(oi, m, ci, ai, q, amt)))) }
```

Q5: cid, cname, city of customers who placed at least one order

```
{ <ci, cn, ct> | ∃ r ∃ s (Customer(ci, cn, ct, r, s) ∧
                  □ oi □ m □ ai □ q □ amt (Orders(oi, m, ci, ai, q, amt))) }
```

Q: What is SQL? Explain DDL commands (CREATE, ALTER, DROP) with syntax and examples.
Asked: 2081 Ashwin (Back Exam) | Marks: [2+6]

Answer

SQL — Structured Query Language

SQL (Structured Query Language) is the standard language for relational database management systems. It was first developed by IBM (System R) in the 1970s and standardized by ANSI/ISO. SQL is both declarative (specifies what, not how) and comprehensive — it covers all aspects of database interaction.

SQL Sub-languages:

Sub-language	Full Name	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME	Define and modify database structure (schemas)
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE	Retrieve and modify data in tables
DCL	Data Control Language	GRANT, REVOKE	Control access permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

DDL Commands

1. CREATE TABLE

Creates a new table (relation) in the database with its column definitions, data types, and constraints.

```
CREATE TABLE Department (
    dept_id    INT           PRIMARY KEY,
    dept_name  VARCHAR(50)   NOT NULL,
    location   VARCHAR(100)
);

CREATE TABLE Employee (
    empid      INT           PRIMARY KEY,
    name       VARCHAR(100)  NOT NULL,
    dept_id    INT,
    salary     DECIMAL(10,2) CHECK (salary >= 0),
    hiredate   DATE,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
        ON DELETE SET NULL
```

```
        ON UPDATE CASCADE
);
```

2. ALTER TABLE

Modifies an existing table structure — adds/drops columns, modifies data types, adds/drops constraints.

```
-- Add a new column:
ALTER TABLE Employee ADD COLUMN email VARCHAR(100);

-- Modify data type of existing column:
ALTER TABLE Employee MODIFY COLUMN salary DECIMAL(12,2);

-- Drop a column:
ALTER TABLE Employee DROP COLUMN email;

-- Add a constraint:
ALTER TABLE Employee ADD CONSTRAINT chk_sal CHECK (salary > 0);

-- Drop a constraint:
ALTER TABLE Employee DROP CONSTRAINT chk_sal;

-- Add a foreign key:
ALTER TABLE Employee ADD FOREIGN KEY (dept_id) REFERENCES Department(dept_id);
```

3. DROP TABLE

Permanently removes a table (structure and all data) from the database. Cannot be undone.

```
DROP TABLE Employee;         -- drops Employee table and all its data

-- With CASCADE: also drops all dependent objects (views, FKs referencing this
table)
DROP TABLE Department CASCADE;

-- Safe version — no error if table doesn't exist:
DROP TABLE IF EXISTS Employee;
```

4. TRUNCATE TABLE

Removes all rows from a table but keeps the table structure. Faster than DELETE (no WHERE clause, minimal logging).

```
TRUNCATE TABLE Employee;   -- removes all rows; table structure remains
-- Cannot use WHERE clause; resets AUTO_INCREMENT counter
```

5. CREATE INDEX

```
CREATE INDEX idx_salary ON Employee(salary);  -- index for faster salary queries
CREATE UNIQUE INDEX idx_email ON Employee(email);  -- enforces uniqueness
```

6. CREATE DATABASE / USE

```
CREATE DATABASE CompanyDB;
USE CompanyDB;
```

Q: Write SQL queries on Employee(empid, name, dept_id, salary, hiredate) and Department(dept_id, dept_name, location).
Asked: 2081 Ashwin; 2080 Ashwin; 2079 Ashwin/Chaitra; 2075 Baisakh — multiple years | Marks: [4–6]

Answer

SQL Queries — Employee & Department

Q1: Employees with salary > average salary of their own department (correlated subquery)

```
SELECT e1.name, e1.salary, e1.dept_id
FROM   Employee e1
WHERE  e1.salary > (
    SELECT AVG(e2.salary)
    FROM   Employee e2
    WHERE  e2.dept_id = e1.dept_id   -- correlated: references outer query's
dept_id
);
```

Q2: For each dept: dept_name, total salary, avg salary; only where total salary > 500000

```
SELECT d.dept_name,
       SUM(e.salary)  AS total_salary,
       AVG(e.salary)  AS avg_salary,
       COUNT(e.empid) AS num_employees
FROM   Department d
JOIN   Employee e ON d.dept_id = e.dept_id
GROUP  BY d.dept_id, d.dept_name
HAVING SUM(e.salary) > 500000
ORDER  BY total_salary DESC;
```

Q3: Employees who work in same location as department 'Research'

```
-- Method 1: Subquery
SELECT e.name, e.salary, d.dept_name, d.location
FROM   Employee e
JOIN   Department d ON e.dept_id = d.dept_id
WHERE  d.location = (
    SELECT location FROM Department WHERE dept_name = 'Research'
);

-- Method 2: Self-join on Department
SELECT e.name, d.dept_name
FROM   Employee e
JOIN   Department d  ON e.dept_id = d.dept_id
JOIN   Department d2 ON d.location = d2.location
WHERE  d2.dept_name = 'Research'
AND    d.dept_name  <> 'Research';
```

Q4: Employees with salary between 30000 and 50000 in 'Finance' or 'IT' dept

```
SELECT e.name, e.salary, d.dept_name
FROM   Employee e
JOIN   Department d ON e.dept_id = d.dept_id
WHERE  e.salary BETWEEN 30000 AND 50000
AND    d.dept_name IN ('Finance', 'IT');
```

Q5: Department with highest average salary

```
-- Method 1: ORDER BY + LIMIT
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM   Department d
JOIN   Employee e ON d.dept_id = e.dept_id
GROUP  BY d.dept_id, d.dept_name
ORDER  BY avg_salary DESC
LIMIT  1;

-- Method 2: Subquery with HAVING (portable across SQL standards)
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM   Department d
JOIN   Employee e ON d.dept_id = e.dept_id
GROUP  BY d.dept_id, d.dept_name
HAVING AVG(e.salary) = (
    SELECT MAX(avg_sal)
    FROM  (SELECT AVG(salary) AS avg_sal
           FROM Employee GROUP BY dept_id) AS sub
);
```

Q6: Names and salary of employees earning more than ANY employee in dept 'IT'

```
SELECT name, salary
FROM   Employee
WHERE  salary > ANY (
    SELECT e.salary
    FROM   Employee e
    JOIN   Department d ON e.dept_id = d.dept_id
    WHERE  d.dept_name = 'IT'
);

-- Note: > ANY means 'greater than at least one' i.e., > MIN of IT salaries
-- > ALL would mean 'greater than EVERY IT employee' (> MAX of IT salaries)
```

Q7: Employees in 'Sales' or 'Marketing' departments with salary > 50000

```
SELECT e.name, e.salary, d.dept_name
FROM   Employee e
JOIN   Department d ON e.dept_id = d.dept_id
WHERE  d.dept_name IN ('Sales', 'Marketing')
AND    e.salary > 50000;
```

Q8: Department names and count of employees; include depts with 0 employees

```
SELECT d.dept_name, COUNT(e.empid) AS num_employees
FROM   Department d
LEFT   JOIN Employee e ON d.dept_id = e.dept_id
```

```
GROUP  BY d.dept_id, d.dept_name
ORDER  BY num_employees DESC;

-- LEFT JOIN ensures departments with no employees show COUNT = 0
```

Q9: Display deptno, dname, total employees, average salary grouped by dept

```
SELECT d.dept_id, d.dept_name,
       COUNT(e.empid)    AS total_employees,
       COALESCE(AVG(e.salary), 0) AS avg_salary
FROM   Department d
LEFT   JOIN Employee e ON d.dept_id = e.dept_id
GROUP  BY d.dept_id, d.dept_name;
```

Q10: Employees hired in last 5 years with salary > average salary

```
SELECT name, salary, hiredate
FROM   Employee
WHERE  hiredate >= DATE_SUB(CURDATE(), INTERVAL 5 YEAR)
AND    salary > (SELECT AVG(salary) FROM Employee);

-- DATE_SUB(CURDATE(), INTERVAL 5 YEAR) = today minus 5 years
```

Q11: Employee names and dept names (natural join equivalent)

```
SELECT e.name, d.dept_name, d.location
FROM   Employee e
JOIN   Department d ON e.dept_id = d.dept_id;
```

Q12: Total salary per department

```
SELECT d.dept_name, SUM(e.salary) AS total_salary
FROM   Employee e
JOIN   Department d ON e.dept_id = d.dept_id
GROUP  BY d.dept_id, d.dept_name;
```

Q: Write SQL queries on Customer(cid,cname,city,rating,sal), Agent(aid,aname,city,percent), Orders(oid,month,cid,aid,qty,amount).
Asked: 2082 Kartik; 2078; 2075 Bhadra; 2074; 2073; 2071; 2070 (all years) | Marks: [4–8]

Answer

SQL Queries — Customer / Agent / Orders

Q1: Find customers with rating>7, age 28-35 whose name starts with 'A'

(Using extended schema with 'age' column from 2082 Kartik paper)

```
-- Schema: Customer(cid, cname, age, rating, sal, city)
```

```
SELECT cid, cname, age, rating, city
FROM   Customer
WHERE  rating > 7
AND    age BETWEEN 28 AND 35
AND    cname LIKE 'A%';
```

Q2: Find 5 oldest customers; display cid, cname, city, age; youngest first within top-5

```
-- 5 oldest means top 5 by highest age; then sort youngest-first (ASC) within
those 5
SELECT cid, cname, city, age
FROM (
    SELECT cid, cname, city, age
    FROM   Customer
    ORDER  BY age DESC
    LIMIT  5
) AS top5
ORDER BY age ASC;

-- Outer query sorts the 5 oldest ascending (youngest-of-the-5 first)
```

Q3: Agent names for customers from 'London' (without JOIN keyword — subquery)

```
SELECT aname
FROM   Agent
WHERE  aid IN (
    SELECT aid
    FROM   Orders
    WHERE  cid IN (
        SELECT cid
        FROM   Customer
        WHERE  city = 'London'
    )
);
```

Q4: Names of agents who have NOT placed any order for customer 'Nick'

```
SELECT aname
FROM   Agent
WHERE  aid NOT IN (
    SELECT o.aid
    FROM   Orders o
    JOIN   Customer c ON o.cid = c.cid
    WHERE  c.cname = 'Nick'
);
```

Q5: Display cid, cname, number_of_orders, average_quantity for all customers

```
SELECT c.cid, c.cname,
       COUNT(o.oid)  AS number_of_orders,
       AVG(o.qty)    AS average_quantity
FROM   Customer c
LEFT   JOIN Orders o ON c.cid = o.cid
GROUP  BY c.cid, c.cname
ORDER  BY number_of_orders DESC;
```

```
-- LEFT JOIN includes customers with 0 orders (COUNT=0, AVG=NULL)
```

Q6: Customers who have ordered from ALL agents based in 'London' (Relational Division in SQL)

```
-- Division: customers who ordered from EVERY London agent
-- Method: NOT EXISTS (NOT EXISTS) pattern
SELECT c.cid, c.cname
FROM   Customer c
WHERE  NOT EXISTS (
    -- There is NO London agent...
    SELECT a.aid FROM Agent a WHERE a.city = 'London'
    AND NOT EXISTS (
        -- ...for whom this customer has NOT placed an order
        SELECT o.oid FROM Orders o
        WHERE  o.cid = c.cid AND o.aid = a.aid
    )
);
```

Q7: Total orders and total amount per agent

```
SELECT a.aname,
       COUNT(o.oid)   AS total_orders,
       SUM(o.amount)  AS total_amount
FROM   Agent a
LEFT   JOIN Orders o ON a.aid = o.aid
GROUP  BY a.aid, a.aname;
```

Q8: Customers who have placed orders in January AND in February (both months)

```
SELECT DISTINCT c.cname
FROM   Customer c
WHERE  c.cid IN (SELECT cid FROM Orders WHERE month='Jan')
AND    c.cid IN (SELECT cid FROM Orders WHERE month='Feb');
```

Q9: Find customers and agents in same city

```
SELECT c.cname, a.aname, c.city
FROM   Customer c
JOIN   Agent a ON c.city = a.city;
```

Q10: Agents with total order amount > 100000

```
SELECT a.aname, SUM(o.amount) AS total_amount
FROM   Agent a
JOIN   Orders o ON a.aid = o.aid
GROUP  BY a.aid, a.aname
HAVING SUM(o.amount) > 100000;
```

Q: What is a view? Why are views used? Explain how to create, use, list, and drop views in SQL with examples.
Asked: 2081 Ashwin; 2079 Chaitra; 2073 Bhadra | Marks: [4]

Answer

Views in SQL

A view is a virtual table defined by a SELECT query stored in the database with a name. It does not store data physically — it is computed from base tables each time it is queried. A view looks and behaves like a real table for query purposes.

Why are Views Used?

- Simplicity: Complex queries involving many joins can be encapsulated in a view for repeated use
- Security: Restrict access to sensitive columns — show only name and dept_name, hide salary and SSN
- Logical data independence: If the underlying schema changes, update only the view definition
- Customization: Different users see different virtual tables tailored to their needs
- Read-only access: Views can prevent INSERT/UPDATE/DELETE on sensitive tables
- Query reuse: Avoid repeating complex query logic

SQL View Operations

1. CREATE VIEW

```
-- Basic view: employee name and dept_name for employees in 'HR' dept
CREATE VIEW HR_Employees AS
SELECT e.name, d.dept_name
FROM   Employee e
JOIN   Department d ON e.dept_id = d.dept_id
WHERE  d.dept_name = 'HR';

-- View with empid, name, salary, dept_name
CREATE VIEW Employee_Info AS
SELECT e.empid, e.name, e.salary, d.dept_name
FROM   Employee e
JOIN   Department d ON e.dept_id = d.dept_id;

-- View with aggregation: dept summary
CREATE VIEW Dept_Summary AS
SELECT d.dept_name,
       COUNT(e.empid) AS num_employees,
       AVG(e.salary)  AS avg_salary
FROM   Department d
LEFT   JOIN Employee e ON d.dept_id = e.dept_id
GROUP  BY d.dept_id, d.dept_name;
```

2. Querying a View

```
-- Query a view just like a table:
SELECT * FROM HR_Employees;

SELECT name FROM Employee_Info WHERE salary > 50000;
```

```
SELECT dept_name, avg_salary FROM Dept_Summary ORDER BY avg_salary DESC;
```

3. WITH CHECK OPTION

```
-- Ensures INSERT/UPDATE through view respect the view's WHERE clause:
CREATE VIEW High_Earners AS
SELECT * FROM Employee WHERE salary > 50000
WITH CHECK OPTION;

-- Now, inserting/updating through this view is checked:
-- INSERT INTO High_Earners VALUES (e99, 'Eve', d01, 40000) -- REJECTED (salary <
50000)
```

4. DROP VIEW

```
DROP VIEW HR_Employees;
DROP VIEW IF EXISTS Employee_Info;
```

5. Listing Views (database-specific)

```
-- MySQL:
SHOW FULL TABLES WHERE Table_type = 'VIEW';

-- PostgreSQL:
SELECT viewname FROM pg_views WHERE schemaname = 'public';

-- Standard SQL / information schema:
SELECT TABLE_NAME FROM INFORMATION_SCHEMA.VIEWS;
```

View Updatability

A view is updatable (allows INSERT/UPDATE/DELETE) only if it meets certain conditions:

- Does NOT use DISTINCT, GROUP BY, HAVING, UNION, or aggregate functions
- Does NOT use subqueries in SELECT list
- Every NOT NULL column of the base table is accessible through the view
- View maps to a single base table (no joins in simpler implementations)

Q: What is a trigger? Describe active and passive database constraints. Write an SQL trigger example.

Asked: 2082 Kartik; 2072 Ashwin; 2072 Magh | Marks: [4]

Answer

Triggers in SQL

A trigger is a stored program that automatically executes (fires) in response to specific events (INSERT, UPDATE, DELETE) on a specified table. Triggers implement active database capabilities — the database automatically responds to data changes.

Active vs Passive Constraints

Aspect	Passive Constraints	Active Constraints (Triggers)
Definition	Checked only when explicitly violated by an operation	Automatically execute code in response to events
Enforcement	DBMS checks constraint at time of DML operation	DBMS fires trigger code automatically
Examples	PRIMARY KEY, FOREIGN KEY, CHECK, NOT NULL, UNIQUE	AFTER INSERT trigger, BEFORE UPDATE trigger
Complexity	Simple, declarative — no procedural code	Complex — can contain procedural SQL code
Flexibility	Fixed rule (can't branch or call procedures)	Can implement complex business rules, logging, cascades
Performance	Lightweight check	Heavier — executes additional SQL code
Reusability	Constraint reused automatically	Trigger fires automatically for any matching DML

Trigger Syntax

```
CREATE TRIGGER trigger_name
    {BEFORE | AFTER} {INSERT | UPDATE | DELETE}
    ON table_name
    FOR EACH ROW
BEGIN
    -- trigger body: SQL statements
    -- NEW.column_name  = new value (for INSERT/UPDATE)
    -- OLD.column_name  = old value (for UPDATE/DELETE)
END;
```

Example 1: Salary Update Validation (BEFORE UPDATE trigger)

```
DELIMITER //
CREATE TRIGGER trg_check_salary
    BEFORE UPDATE ON Employee
    FOR EACH ROW
BEGIN
    IF NEW.salary < OLD.salary THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be decreased!';
    END IF;
    IF NEW.salary > OLD.salary * 2 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary increase cannot exceed 100%!';
    END IF;
END //
DELIMITER ;
```

Example 2: Audit Log Trigger (AFTER UPDATE trigger)

```
-- First create audit table:
CREATE TABLE SalaryAudit (
    audit_id    INT AUTO_INCREMENT PRIMARY KEY,
    empid       INT,
```

```
    old_salary  DECIMAL(10,2),
    new_salary  DECIMAL(10,2),
    changed_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
    changed_by  VARCHAR(50) DEFAULT USER()
);

DELIMITER //
CREATE TRIGGER trg_salary_audit
    AFTER UPDATE ON Employee
    FOR EACH ROW
BEGIN
    IF NEW.salary <> OLD.salary THEN
        INSERT INTO SalaryAudit(empid, old_salary, new_salary)
        VALUES (OLD.empid, OLD.salary, NEW.salary);
    END IF;
END //
DELIMITER ;
```

Example 3: AFTER INSERT trigger — Automatically create a profile entry

```
DELIMITER //
CREATE TRIGGER trg_after_employee_insert
    AFTER INSERT ON Employee
    FOR EACH ROW
BEGIN
    INSERT INTO EmployeeProfile(empid, join_year, status)
    VALUES (NEW.empid, YEAR(NEW.hiredate), 'Active');
END //
DELIMITER ;
```

Q: Define Embedded SQL. Explain with a C program example using EXEC SQL, SQLCA, host variables, and cursors.
Asked: 2073 Magh (Back Exam); 2070 Magh (Back Exam) | Marks: [4–6]

Answer

Embedded SQL

Embedded SQL refers to SQL statements that are written directly inside a host programming language (such as C, C++, Java, COBOL, FORTRAN) within the same source file. The combined source is pre-processed by a pre-compiler that translates SQL statements into function calls before standard compilation.

Why Embedded SQL?

- SQL alone cannot express complete applications (no loops, conditionals, I/O)
- Host language provides procedural control flow; SQL provides data access
- Allows row-by-row processing of query results using cursors
- Enables complex database applications beyond simple ad-hoc queries

Key Concepts

Concept	Description
EXEC SQL	Prefix that identifies an embedded SQL statement to the pre-compiler
Host variable	Variable in the host language used to exchange data with SQL; prefixed with : in SQL statements
SQLCA (SQL Communication Area)	Data structure that holds status information after each SQL statement execution; SQLCODE indicates success/failure
Cursor	A named pointer to the result set of a query; used to iterate through multi-row results one row at a time
DECLARE SECTION	Block that declares host variables: EXEC SQL BEGIN/END DECLARE SECTION
Pre-compiler	Translates EXEC SQL statements into host language function calls (e.g., Oracle Pro*C pre-compiler)

Embedded SQL in C — Complete Example

```
#include <stdio.h>
#include <string.h>

/* — 1. SQLCA — SQL Communication Area — */
EXEC SQL INCLUDE SQLCA;     /* provides sqlca.sqlcode, sqlca.sqlerrm */

/* — 2. Host Variable Declaration Section — */
EXEC SQL BEGIN DECLARE SECTION;
    int    h_empid;
    char   h_name[101];
    float  h_salary;
    int    h_dept_id;
    char   h_dept_name[51];
    float  h_threshold;
EXEC SQL END DECLARE SECTION;

int main() {
    /* — 3. Connect to database — */
    EXEC SQL CONNECT TO 'CompanyDB' USER 'admin' USING 'password';
    if (sqlca.sqlcode != 0) {
        printf("Connection failed: %d\n", sqlca.sqlcode);
        return 1;
    }

    /* — 4. Single-row query (INTO clause) — */
    h_empid = 101;
    EXEC SQL SELECT name, salary, dept_id
             INTO   :h_name, :h_salary, :h_dept_id
             FROM   Employee
             WHERE  empid = :h_empid;

    if (sqlca.sqlcode == 0) {
        printf("Employee: %s, Salary: %.2f\n", h_name, h_salary);
    } else {
        printf("Not found. SQLCODE = %d\n", sqlca.sqlcode);
    }
```

```
    /* ── 5. Multi-row query using CURSOR ── */
    h_threshold = 40000.0;

    /* Declare cursor — defines the query */
    EXEC SQL DECLARE emp_cursor CURSOR FOR
        SELECT e.empid, e.name, e.salary, d.dept_name
        FROM   Employee e
        JOIN   Department d ON e.dept_id = d.dept_id
        WHERE  e.salary > :h_threshold
        ORDER  BY e.salary DESC;

    /* Open cursor — executes the query */
    EXEC SQL OPEN emp_cursor;

    printf("\nEmployees with salary > %.0f:\n", h_threshold);
    printf("%-6s %-20s %10s  %s\n",
           "ID", "Name", "Salary", "Department");

    /* Fetch cursor — retrieves rows one by one */
    while (1) {
        EXEC SQL FETCH emp_cursor
                 INTO :h_empid, :h_name, :h_salary, :h_dept_name;

        if (sqlca.sqlcode == 100) break;  /* 100 = no more rows (NOT FOUND) */
        if (sqlca.sqlcode  <  0) {         /* negative = error */
            printf("Error: %d\n", sqlca.sqlcode);
            break;
        }
        printf("%-6d %-20s %10.2f  %s\n",
               h_empid, h_name, h_salary, h_dept_name);
    }

    /* Close cursor — releases resources */
    EXEC SQL CLOSE emp_cursor;

    /* ── 6. DML with host variables ── */
    h_empid  = 105;
    h_salary = 55000.0;
    EXEC SQL UPDATE Employee
             SET    salary = :h_salary
             WHERE  empid  = :h_empid;

    if (sqlca.sqlcode == 0) {
        EXEC SQL COMMIT;
        printf("\nSalary updated successfully.\n");
    } else {
        EXEC SQL ROLLBACK;
        printf("Update failed: %d\n", sqlca.sqlcode);
    }

    EXEC SQL DISCONNECT;
    return 0;
}
```

Cursor Operations Summary

Operation	Syntax	Purpose
DECLARE	EXEC SQL DECLARE cur CURSOR FOR SELECT ...	Define cursor and its query

OPEN	EXEC SQL OPEN cur	Execute the cursor's SELECT query; position before first row
FETCH	EXEC SQL FETCH cur INTO :var1, :var2	Retrieve next row into host variables
CLOSE	EXEC SQL CLOSE cur	Release cursor resources

SQLCA Status Codes:

- sqlca.sqlcode = 0: Operation successful
- sqlca.sqlcode = 100 (or +100): No data found (end of cursor, or SELECT INTO found no rows)
- sqlca.sqlcode < 0: Error — negative codes are error-specific

---

Q: Find the candidate key(s) of R(A,B,C,D,E) with functional dependencies: BC→AD, D→E, AC→B.

Asked: 2082 Kartik (Back Exam) | Marks: [4]

Answer

Method: Compute Attribute Closures

To find candidate keys, we compute the closure of various subsets of attributes. A set X is a candidate key if X+ = {A,B,C,D,E} (i.e., X determines all attributes) and no proper subset of X has this property.

Given Functional Dependencies:

- BC → AD
- D → E
- AC → B

Step 1: Identify attributes that never appear on RHS of any FD

These attributes must be in every candidate key (they cannot be derived from anything).

- A: appears on RHS of BC→AD — so A can be derived
- B: appears on RHS of BC→AD and AC→B — so B can be derived
- C: never on RHS — C MUST be in every candidate key
- D: appears on RHS of BC→AD — can be derived
- E: appears on RHS of D→E — can be derived

Conclusion: C must be in every candidate key. We need to find what to combine with C.

Step 2: Try AC

```
Closure of AC: {AC}+
Start:  {A, C}
AC → B  →  add B:   {A, B, C}
BC → AD →  add A, D (A already in):  {A, B, C, D}
D  → E  →  add E:   {A, B, C, D, E}

{AC}+ = {A, B, C, D, E} = R   ✓  AC is a superkey
```

Check minimality: Is A alone a superkey? No (A cannot derive B, C, D, E with given FDs). Is C alone a superkey? No. So AC is a candidate key.

Step 3: Try BC

```
Closure of BC: {BC}+
Start:  {B, C}
BC → AD →  add A, D:  {A, B, C, D}
AC → B  →  B already in:  {A, B, C, D}
D  → E  →  add E:  {A, B, C, D, E}

{BC}+ = {A, B, C, D, E} = R   ✓  BC is a superkey
```

Check minimality: B alone: {B}+ = {B} — cannot derive C,A,D,E. C alone: {C}+ = {C}. So BC is a candidate key.

Step 4: Try other combinations with C

```
{CD}+:  Start {C, D}
D → E:  add E:  {C, D, E}
No other FD applies with only C,D,E on LHS.
{CD}+ = {C, D, E} ≠ R   ✗  Not a superkey

{CE}+:  Start {C, E}
No FD has only C or E or CE on LHS.
{CE}+ = {C, E} ≠ R   ✗  Not a superkey
```

Result

The candidate keys of R(A,B,C,D,E) are:

- AC — (closure covers all attributes; A and C individually are not superkeys)
- BC — (closure covers all attributes; B and C individually are not superkeys)

Both AC and BC are candidate keys. There is no other candidate key (all other subsets containing C either give supersets of these or don't cover R).

Q: Write advanced SQL queries: RANK/DENSE_RANK window functions, correlated subqueries, and complex filtering.

Asked: 2081 Chaitra (Regular Exam); 2081 Ashwin (Back Exam) | Marks: [3+3]

Answer

Advanced SQL Features

Q1: Employees hired before current year with salary rank in top 3 per department (Window Functions)

```
-- Using RANK() / DENSE_RANK() window function
SELECT ename, deptno, salary, hiredate, sal_rank
FROM (
    SELECT e.ename, e.deptno, e.salary, e.hiredate,
           DENSE_RANK() OVER (
               PARTITION BY e.deptno          -- rank within each dept separately
               ORDER BY e.salary DESC          -- rank by salary descending
           ) AS sal_rank
    FROM Employee e
    WHERE YEAR(e.hiredate) < YEAR(CURDATE())   -- hired before current year
```

```
) ranked
WHERE sal_rank <= 3
ORDER BY deptno, sal_rank;

-- DENSE_RANK vs RANK:
--   RANK(): gaps in rank when ties exist (1,2,2,4)
--   DENSE_RANK(): no gaps (1,2,2,3)
```

Q2: Employees with salary > average salary of their department (correlated subquery)

```
SELECT e1.ename, e1.deptno, e1.salary,
       (SELECT AVG(e2.salary)
        FROM Employee e2
        WHERE e2.deptno = e1.deptno) AS dept_avg
FROM Employee e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.deptno = e1.deptno   -- correlated: references outer e1.deptno
);
```

Q3: Retrieve employees above average salary (non-correlated subquery)

```
SELECT name, salary
FROM   Employee
WHERE  salary > (SELECT AVG(salary) FROM Employee);
-- This is NOT correlated — inner query doesn't reference outer query
```

Q4: Using EXISTS and NOT EXISTS

```
-- Employees who have at least one peer in same dept earning more (using EXISTS):
SELECT e1.name, e1.salary, e1.dept_id
FROM   Employee e1
WHERE  EXISTS (
    SELECT 1 FROM Employee e2
    WHERE  e2.dept_id = e1.dept_id
    AND    e2.salary  > e1.salary
);

-- Departments with NO employees (using NOT EXISTS):
SELECT d.dept_name
FROM   Department d
WHERE  NOT EXISTS (
    SELECT 1 FROM Employee e WHERE e.dept_id = d.dept_id
);
```

Q5: ROW_NUMBER() — assign sequential number within partition

```
SELECT name, dept_id, salary,
       ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS row_num
FROM Employee;
-- ROW_NUMBER: always unique, no ties (1,2,3,4 even if salaries are equal)
```

SUMMARY TABLE — Chapter 3 Question Coverage

This table maps all Chapter 3 questions from TU-IOE BCT examination papers (2070–2082) to the answers in this document.

No.	Question / Topic	Asked In	Marks
C1	Relational Model: relation, schema, instance, tuple, attribute, domain, degree, cardinality	2074, 2072 Magh, 2080	[2+4]
C2	Integrity Constraints (domain, key, entity, referential)	2080 Ashwin	[4]
C3	Relational Algebra — all 8 operations with examples	2081 Chaitra, 2079 Ashwin, 2075 Bhadra, 2070-2077 all	[2+4 to 6]
C4	RA vs Relational Calculus (TRC and DRC); RA vs TRC vs DRC	2079 Chaitra, 2075 Baisakh	[4]
C5	RA expressions: Employee & Department tables	2080, 2079, 2078, 2073, all	[4-6]
C6	RA expressions: Customer, Agent, Orders tables	2078, 2077, 2075, 2070 all	[4]
C7	Tuple Relational Calculus (TRC) — definition and expressions	2079 Jestha, 2078 Chaitra, 2077, 2076, 2073, 2071	[4]
C8	Domain Relational Calculus (DRC) — definition and expressions	2079 Jestha, 2078 Chaitra, 2077, 2076	[4]
C9	SQL DDL — CREATE, ALTER, DROP, TRUNCATE with syntax & examples	2081 Ashwin	[2+6]
C10	SQL queries: Employee & Department — subqueries, GROUP BY, HAVING	2081 Ashwin, 2080, 2079, 2075 Baisakh	[4-6]
C11	SQL queries: Customer, Agent, Orders — aggregation, division, subquery	2082, 2078, 2075, 2074, 2073, 2071, 2070	[4-8]
C12	SQL Views — CREATE VIEW, WITH CHECK OPTION, DROP VIEW	2081 Ashwin, 2079, 2073 Bhadra	[4]
C13	SQL Triggers — active vs passive constraints, trigger syntax	2082, 2072 Ashwin/Magh	[4]
C14	Embedded SQL — EXEC SQL, SQLCA, cursors, host variables	2073 Magh, 2070 Magh	[4-6]
C15	Candidate Key from Functional Dependencies	2082 Kartik	[4]
C16	Advanced SQL: Ranking (RANK/DENSE_RANK), correlated subqueries, LIMIT/FETCH	2082, 2081 Chaitra	[3+3]

Note: SQL syntax uses MySQL dialect throughout. PostgreSQL and Oracle differences are minor (e.g., LIMIT vs FETCH FIRST, CURDATE() vs SYSDATE). The RA notation uses standard mathematical symbols: σ=Selection, π=Projection, ⋈=Natural Join, ÷=Division, ρ=Rename, ∪=Union, −=Set Difference, ×=Cartesian Product.

— End of Chapter 3 Answer Bank —